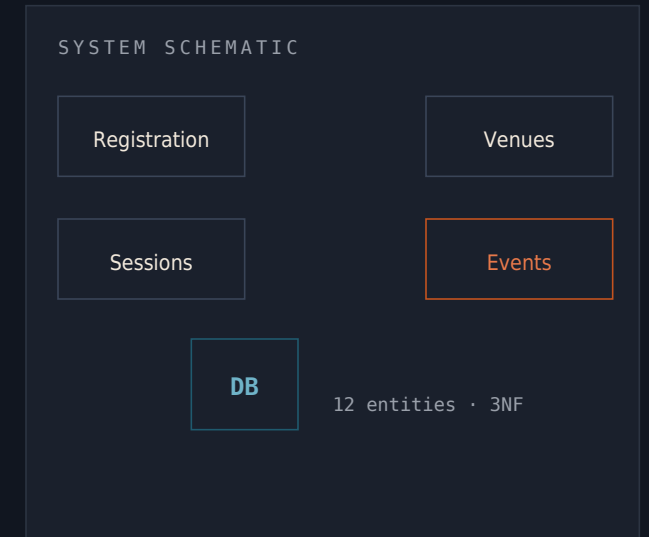


ACADEMIC PROJECT

Database System for Event Registration & Venue Scheduling



A relational DBMS designed for seminars, workshops and cultural events — normalized to 3NF, validated, indexed, reportable.

PRESENTED BY

Devesh

25wu0101030

Sarthak

25wu0101025

Dhruv

25wu0101031

UNIVERSITY / CLASS / MENTOR

Woxsen University

B.Tech CSE · CSE Tigers

Semester 3

Mentor: Dr. Kiran Mayee Adavala

Problem, objectives, **scope** and users.

PROBLEM STATEMENT

Institutions run seminars, workshops and cultural fests on paper, spreadsheets and noticeboards. The result: lost entries, double-booked venues, overcrowded halls, no attendance audit trail, and unreliable payments, certificates and feedback.

OBJECTIVES

01 Reliable relational schema
Modeled on real workflows, normalized to 3NF.

02 Manage events & sessions
Speakers, rooms and timings are first-class.

03 Prevent venue conflicts
No double-bookings or capacity overruns.

04 Manage registrations
One participant per event, no duplicates.

05 Track attendance & payments
Per-session attendance, per-registration payments.

06 Automate certificate eligibility
Issued only after the attendance threshold.

07 Generate reports, keep integrity
Joins, GROUP BY, views; PK/FK rules enforced.

SCOPE

IN SCOPE

- ✓ Events, categories & sessions
- ✓ Venues & capacity checks
- ✓ Participant registration
- ✓ Payments & attendance
- ✓ Certificates & feedback
- ✓ Reports & views

FUTURE / OUT OF SCOPE

- Online payment gateway
- Email & SMS notifications
- QR-based attendance
- Mobile app · cloud deployment

INTENDED USERS

01 Administrator

Owns configuration: creates events, registers venues, sets categories, manages speakers, issues organisation-wide reports.

02 Participant

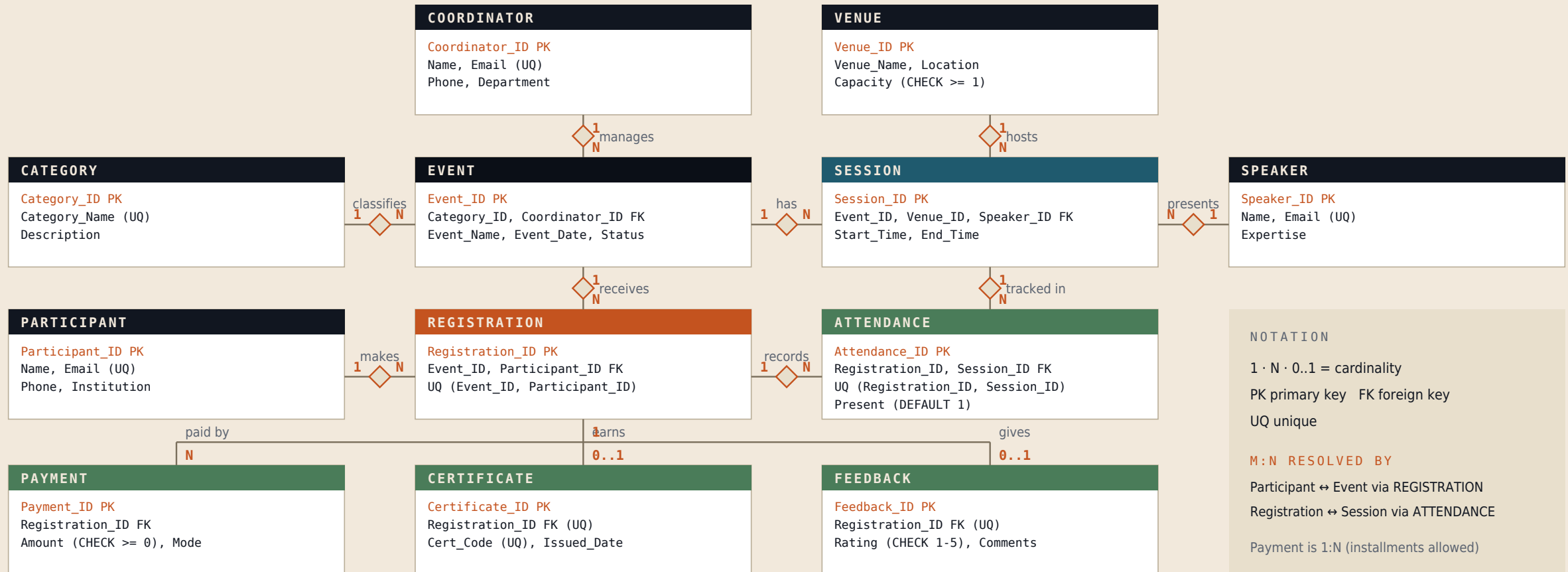
The end-user: browses events, registers and pays, tracks status, submits feedback and downloads the certificate.

03 Coordinator

Events-desk operator: schedules sessions at venues, assigns speakers, records attendance, marks certificate eligibility.

Entity-Relationship model — 12 entities, 12 relationships.

EVENT and REGISTRATION are the hubs. Every attribute shown appears unchanged in the relational schema on slide 4.



PARTICIPATION

TOTAL (mandatory): Event → Category & Coordinator · Session → Event & Venue · Registration → Event & Participant · Attendance → Registration & Session · Payment → Registration.

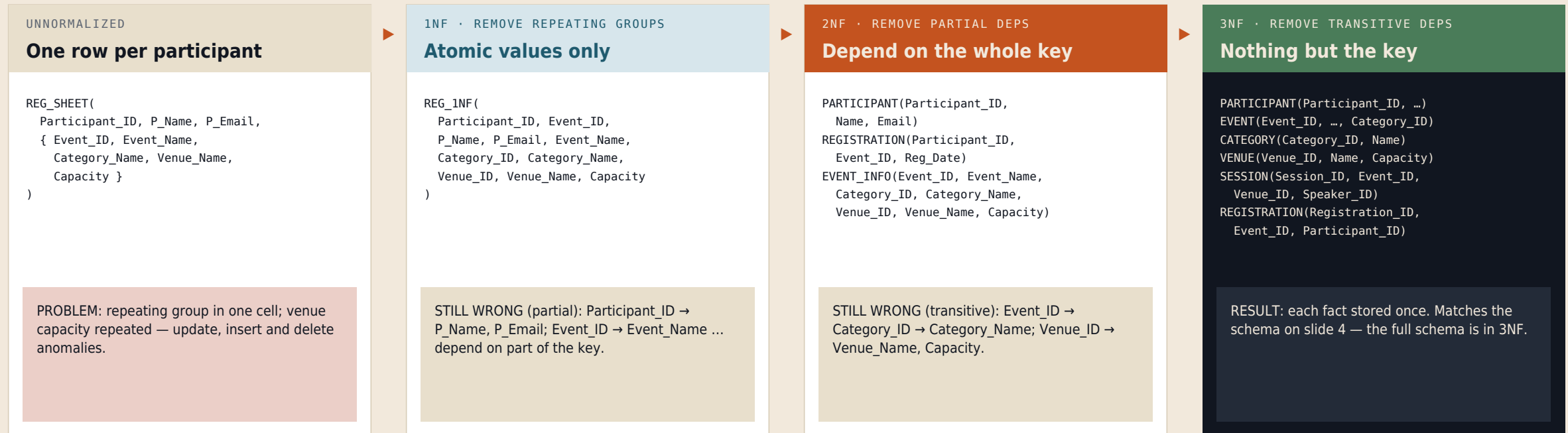
PARTIAL (optional): Venue, Speaker, Category, Coordinator may have no children yet · Certificate and Feedback are 0..1 per Registration.

From entities to tables.

Every table has a primary key, foreign keys are explicit, and constraints are marked inline. PK FK UQ NN = not null.

CATEGORY 01 Category_ID PK Category_Name UQ NN Description	EVENT 02 Event_ID PK Event_Name NN Category_ID FK Coordinator_ID FK Event_Date NN Description Status DEFAULT 'planned'	VENUE 03 Venue_ID PK Venue_Name NN Location Capacity NN CHECK >= 1	SESSION 04 Session_ID PK Event_ID FK Venue_ID FK Speaker_ID FK Start_Time NN End_Time NN Title
COORDINATOR 05 Coordinator_ID PK Name Email UQ Phone Department	SPEAKER 06 Speaker_ID PK Name Email UQ Expertise Bio Contact_Phone	PARTICIPANT 07 Participant_ID PK Name NN Email UQ NN Phone Institution	REGISTRATION 08 Registration_ID PK Event_ID FK Participant_ID FK UNIQUE(Event_ID, Participant_ID) Registration_Date Status DEFAULT 'confirmed'
PAYMENT 09 Payment_ID PK Registration_ID FK Amount CHECK >= 0 Mode Date Status	ATTENDANCE 10 Attendance_ID PK Registration_ID FK Session_ID FK UNIQUE(Registration_ID, Session_ID) Marked_At Present DEFAULT 1	CERTIFICATE 11 Certificate_ID PK Registration_ID FK UQ Issued_Date NN Cert_Code UQ	FEEDBACK 12 Feedback_ID PK Registration_ID FK UQ Rating CHECK 1..5 Comments Submitted_At

One sheet, refined to **Third Normal Form**.



FUNCTIONAL DEPENDENCIES — $A \rightarrow B$ reads "determines": knowing the left side fixes the right side

Participant_ID	→	Name, Email, Phone, Institution	FD • 01
Venue_ID	→	Venue_Name, Location, Capacity	FD • 02
Event_ID	→	Event_Name, Event_Date, Category_ID, Coordinator_ID	FD • 03
Session_ID	→	Event_ID, Venue_ID, Speaker_ID, Start_Time, End_Time	FD • 04
Registration_ID	→	Event_ID, Participant_ID, Registration_Date, Status	FD • 05

Database and queries.

Live on MySQL: tables with PK, FK and constraints · 5+ records per table · SELECTs with joins and aggregates.

01 · SCHEMA

```
CREATE TABLE Venue (
  Venue_ID INT PRIMARY KEY,
  Venue_Name VARCHAR(120) NOT NULL,
  Location VARCHAR(200),
  Capacity INT NOT NULL
  CHECK (Capacity >= 1)
);
```

02 · REFERENTIAL

```
ALTER TABLE Session
  ADD CONSTRAINT fk_session_event
  FOREIGN KEY (Event_ID)
  REFERENCES Event(Event_ID)
  ON DELETE RESTRICT;
```

03 · VIEW

```
CREATE VIEW V_AVAIL_SEATS AS
SELECT S.Session_ID, V.Venue_Name,
  V.Capacity - (SELECT COUNT(*)
    FROM Registration R
    WHERE R.Event_ID = S.Event_ID)
  AS Available
FROM Session S
JOIN Venue V ON V.Venue_ID = S.Venue_ID;
```

04 · FEATURED REPORT – AVAILABLE SEATS PER EVENT

```
SELECT E.Event_Name, V.Venue_Name, V.Capacity,
  COUNT(DISTINCT R.Registration_ID) AS Registered,
  V.Capacity
  - COUNT(DISTINCT R.Registration_ID) AS Available
FROM Event E
JOIN Session S ON S.Event_ID = E.Event_ID
JOIN Venue V ON V.Venue_ID = S.Venue_ID
LEFT JOIN Registration R
  ON R.Event_ID = E.Event_ID
GROUP BY E.Event_ID, E.Event_Name,
  V.Venue_ID, V.Venue_Name, V.Capacity;
```

Algorithms	Soft Skills	Cultural Fest
12 left	28 left	FULL

05 · RETURNING PARTICIPANTS (2+ EVENTS)

```
SELECT P.Name, P.Email
FROM Participant P
JOIN Registration R
  ON R.Participant_ID = P.Participant_ID
WHERE R.Status = 'confirmed'
GROUP BY P.Participant_ID, P.Name, P.Email
HAVING COUNT(*) > 1;
```

CONSTRAINTS USED

PRIMARY KEY	every row addressable
FOREIGN KEY	references that hold
UNIQUE · NOT NULL	required, never twice
CHECK	Amount >= 0 · Capacity >= 1 · Rating 1..5

FIVE BUSINESS RULES

- 1 No venue overlap**
pre-check + trigger
- 2 Capacity not exceeded**
registrations <= capacity
- 3 No duplicate registration**
UNIQUE(Event_ID, Participant_ID)
- 4 Certificate only after threshold**
attendance % >= 75%
- 5 Payment cannot be negative**
CHECK (Amount >= 0)

LIVE DEMO CHECKLIST

- ✓ Tables created with PK / FK / CHECK
- ✓ 5+ records inserted per table
- ✓ SELECT with JOINS & aggregates

Three layers, one schema — a live UI demo.

The presentation layer is thin on purpose; real decisions happen in the application layer and the database.

LAYER 01 · PRESENTATION

Java / Python interface

Forms, lists, search inputs, reports. The schema is hidden behind it.

LAYER 02 · APPLICATION

Logic & validation

CRUD · search · transactions · reporting. Rules are enforced here for fast feedback, and again in the database.

LAYER 03 · DATABASE

MySQL / PostgreSQL

Tables · constraints · indexes · views · triggers. Referential integrity, ACID transactions, CHECK & UNIQUE.

A REGISTER REQUEST, TRACED

01

User submits the registration form

UI posts Event, Participant and payment intent.

02

Validate

Duplicate, capacity, payment ≥ 0 — three pre-checks.

03

Open transaction

INSERT Registration, then Payment — one COMMIT.

04

Database fires constraint checks

FK, UNIQUE, CHECK — a bad insert triggers ROLLBACK.

05

UI confirms or shows an error

Views read the new state for instant feedback.

SIX APPLICATION SCREENS — INSERT · DELETE · VIEW

[Dashboard](#)[Event management](#)[Venue scheduling](#)[Participant registration](#)[Attendance entry](#)[Reports](#)

LIVE UI DEMO — EVERY CHANGE SHOWN IN THE DATABASE

[1 Insert a record in the UI](#)[2 View it in the UI](#)[3 Check it in MySQL](#)[4 Delete the record](#)[5 Confirm in MySQL](#)

Thank you.

Questions & answers

Devesh

25wu0101030

Sarthak

25wu0101025

Dhruv

25wu0101031